

```

rich@LAPTOP-2J0OKKDC:~/Lab1$ ./test.sh
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c
-lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 6/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 21/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 39/48 operators
mul5Sat: 27/30 operators
classifyAdd3: 51/52 operators
floatScaleThreeHalves: 40/60 operators
floatRoundEven: 27/65 operators
float_i2f: 25/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
operator check passed

```

```

operator check passed
Score  Rating  Errors  Function
1      1        0    signMask
2      2        0    bitXor
3      3        0    negativePart
4      4        0    copyByteWithin
4      4        0    logicalShift
4      4        0    swapNibblePairs
4      4        0    secondLowestZeroBit
5      5        0    oddParity
5      5        0    rotateRightBits
5      5        0    roundEvenPow2
5      5        0    midpointTowardFirst
7      7        0    isBetweenEitherOrder
7      7        0    mul5Sat
7      7        0    classifyAdd3
7      7        0    floatScaleThreeHalves
10     10        0    floatRoundEven
10     10        0    float_i2f
10     10        0    bitCount
10     10        0    bitReverse
Total points: 110/110
all checks passed: 110/110
rich@LAPTOP-2J0OKKDC:~/Lab1$ █

```

P1:将数字 1 左移 31 位，使其到达最高位，生成符号位掩码

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f signMask
Score   Rating   Errors   Function
1       1         0       signMask
Total points: 1/1
```

P2: 异或等价于不同为 1。利用德摩根定律，将 $(\sim x \ \& \ y) \mid (x \ \& \ \sim y)$ 中的或运算替换为取反和与运算

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f bitXor
Score   Rating   Errors   Function
2       2         0       bitXor
Total points: 2/2
```

P3: 通过算术右移 31 位获取符号位掩码，利用取反加 1 的补码特性求得相反数，最后用掩码与运算过滤正数

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f negativePart
Score   Rating   Errors   Function
3       3         0       negativePart
Total points: 3/3
```

P4: 先提取源字节并移至目标位置，同时清空原数的目标字节，最后将两者合并

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f copyByteWithin
Score   Rating   Errors   Function
4       4         0       copyByteWithin
Total points: 4/4
```

P5:先算术右移，再用取反后的符号位掩码消除高位多余的 1，实现逻辑右移

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f logicalShift
Score   Rating   Errors   Function
4       4         0       logicalShift
Total points: 4/4
```

P6:将 x 的右移 4 位与左移 4 位的结果相或，并用 $0 \times 0F0F0F0F$ 掩码清除相邻字节的干扰

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f swapNibblePairs
Score  Rating  Errors  Function
   4      4      0      swapNibblePairs
Total points: 4/4
```

P7:先通过 $x | (x + 1)$ 将最低位 0 翻转为 1，再将结果取反并与自身加 1 相与，从而提取出次低位 0 的位置掩码

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f secondLowestZeroBit
Score  Rating  Errors  Function
   4      4      0      secondLowestZeroBit
Total points: 4/4
```

P8:通过分治异或依次折叠相邻 2 位、4 位、8 位、16 位的奇偶性，将 32 位整数的奇偶结果集中到最低位，最后提取并取反

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f oddParity
Score  Rating  Errors  Function
   5      5      0      oddParity
Total points: 5/5
```

P9:通过位运算规范移位量，将原数逻辑右移并掩码清除符号位，再与低位移至高位的原数拼接完成循环右移

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f rotateRightBits
Score  Rating  Errors  Function
   5      5      0      rotateRightBits
Total points: 5/5
```

P10:通过位运算构造对齐掩码与半值，先常规加半值对齐，再判断恰逢中间值且商为奇数时向下调整，实现向偶数舍入

```
rich@LAPTOP-2J00KKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J00KKDC:~/Lab1$ ./btest -f roundEvenPow2
Score   Rating  Errors  Function
   5       5       0    roundEvenPow2
Total points: 5/5
```

P11:通过位运算无溢出计算基础平均值，再提取符号位判断两数大小与异号情况，最后在结果为半整数时向 x 方向微调

```
rich@LAPTOP-2J00KKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J00KKDC:~/Lab1$ ./btest -f midpointTowardFirst
Score   Rating  Errors  Function
   5       5       0    midpointTowardFirst
Total points: 5/5
```

P12:通过位运算无分支比较大小，分别判断 x 是否在 a 到 b 或 b 到 a 的区间内，最后将两种区间方向的结果组合得出判定

```
rich@LAPTOP-2J00KKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J00KKDC:~/Lab1$ ./btest -f isBetweenEitherOrder
Score   Rating  Errors  Function
   7       7       0    isBetweenEitherOrder
Total points: 7/7
```

P13:通过位运算实现无溢出乘 5 并检测溢出，利用符号位选择对应的饱和极值，最后通过掩码无分支地返回正常结果或饱和值

```
rich@LAPTOP-2J00KKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J00KKDC:~/Lab1$ ./btest -f mul5Sat
Score   Rating  Errors  Function
   7       7       0    mul5Sat
Total points: 7/7
```

P14:通过分步累加提取符号位逻辑推演进位与高位状态，据此判断三数之和的正负溢出情况，并用位运算无分支返回对应的 1、-1 或 0 分类结果

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f classifyAdd3
Score  Rating  Errors  Function
   7      7      0      classifyAdd3
Total points: 7/7
```

P15:通过提取符号、阶码和尾数，先处理 NaN 特殊值保留原参；再针对非规格化与规格化数分别乘 1.5，结合向偶数舍入规则调整尾数，处理归一化溢出与阶码上溢转无穷大，最后重组位级结果并保留原符号

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f floatScaleThreeHalves
Score  Rating  Errors  Function
   7      7      0      floatScaleThreeHalves
Total points: 7/7
```

P16:通过提取符号、阶码和尾数，先处理 NaN/无穷大及极值边界情况；再对规格化数提取隐藏位，按指数位移分离高低位，依据向偶数舍入规则调整，最后重组阶码与尾数输出结果

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f floatRoundEven
Score  Rating  Errors  Function
  10    10      0      floatRoundEven
Total points: 10/10
```

P17:先把特殊值（0 和最小负数）挑出来处理，然后记下符号并把负数转正，接着通过左移找到最高位的 1 来确定指数，最后截取小数部分按“向偶数舍入”规则处理并拼回浮点数的位模式

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f float_i2f
Score  Rating  Errors  Function
  10    10      0      float_i2f
Total points: 10/10
```

P18:使用分治法即并行计数来实现

```
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btset -f bitCount
-bash: ./btset: No such file or directory
rich@LAPTOP-2J0OKKDC:~/Lab1$ ./btest -f bitCount
Score  Rating  Errors  Function
  10    10      0      bitCount
Total points: 10/10
```

P19:采用分治策略，按相邻 1 位→相邻 2 位→相邻 4 位→相邻 8 位→相邻 16 位的粒度从小到大依次交换对应分组的位置，逐步完成整个 32 位整数的全位序反转

```
rich@LAPTOP-2J00KKDC:~/Lab1$ make clean && make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c
-lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
rich@LAPTOP-2J00KKDC:~/Lab1$ ./btest -f bitReverse
Score   Rating  Errors  Function
   10      10      0      bitReverse
Total points: 10/10
```